

Assignmnet-3

Group-02

Aashish Sanjel

Dhruv Handa

Honey Shah

Raj Jasani

STEP 1 — Dataset Overview

Purpose of this step

The objective of this step is to load the dataset and conduct an initial structural inspection to understand its size, schema, and basic characteristics. This aligns with the requirement to review column names, data types, and to establish whether the dataset supports the stated project objective.

This step also formally identifies the target variable and the input features based on the project goal. No preprocessing, cleaning, or transformation is performed at this stage; the dataset is examined strictly in its raw form.

Target and input feature definition

Based on the project objective of forecasting **hourly 911 call volumes by department**, the target variable is not directly present in the raw dataset and will be derived in later steps.

- **Target (derived later):**
 - Hourly call count ($\bar{y}$), obtained by aggregating event-level calls by hour.
- **Primary input features:**
 - `timeStamp`: provides the temporal reference required for hourly aggregation.
 - `title`: contains structured text used to derive department labels (EMS, Fire, Traffic).
- **Other features:**
 - Remaining columns (e.g., location and descriptive attributes) provide contextual information and are not required for constructing the core forecasting target.

```
import pandas as pd
import numpy as np

DATASET_PATH = "/content/911.csv"
df = pd.read_csv(DATASET_PATH)

print("DATASET SHAPE (rows, columns):", df.shape)
```

```

print("\nCOLUMN NAMES:")
print(df.columns.tolist())

print("\nDATA TYPES AND NON-NULL COUNTS:")
df.info()

print("\nFIRST 5 ROWS (sample):")
display(df.head())

print("\nTARGET AND INPUT FEATURE IDENTIFICATION (project objective
aligned):")
print("- Target (derived): Hourly call count (y) after aggregating
call events by hour.")
print("- Primary inputs:")
print("  * timeStamp -> temporal index for hourly aggregation")
print("  * title -> structured prefix enables department derivation
(EMS/Fire/Traffic)")
print("- Context features (not required for core target construction):
lat, lng, zip, twp, addr, desc, e")

```

DATASET SHAPE (rows, columns): (663522, 9)

COLUMN NAMES:

```
['lat', 'lng', 'desc', 'zip', 'title', 'timeStamp', 'twp', 'addr',
'e']
```

DATA TYPES AND NON-NULL COUNTS:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 663522 entries, 0 to 663521
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   lat         663522 non-null  float64
1   lng         663522 non-null  float64
2   desc        663522 non-null  object
3   zip         583323 non-null  float64
4   title       663522 non-null  object
5   timeStamp   663522 non-null  object
6   twp         663229 non-null  object
7   addr        663522 non-null  object
8   e           663522 non-null  int64
dtypes: float64(3), int64(1), object(5)
memory usage: 45.6+ MB

```

FIRST 5 ROWS (sample):

```

{"summary":{"\n  \"name\": \"print(\\\"\\\"- Context features (not
required for core target construction): lat, lng, zip, twp, addr,
desc, e\\\"\\\"),\n  \"rows\": 5,\n  \"fields\": [\n    {\n

```

```

\"column\": \"lat\", \n      \"properties\": { \n      \"dtype\":
\"number\", \n      \"std\": 0.08432690431476854, \n      \"min\":
40.116153, \n      \"max\": 40.2978759, \n
\"num_unique_values\": 5, \n      \"samples\": [ \n
40.2580614, \n      40.251492, \n      40.1211818 \n      ], \n
\"semantic_type\": \"\", \n      \"description\": \"\" \n      } \n
}, \n      { \n      \"column\": \"lng\", \n      \"properties\": { \n
\"dtype\": \"number\", \n      \"std\": 0.15316091469337836, \n
\"min\": -75.6033497, \n      \"max\": -75.2646799, \n
\"num_unique_values\": 5, \n      \"samples\": [ \n      -
75.2646799, \n      -75.6033497, \n      -75.3519752 \n
      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"desc\", \n      \"properties\": { \n      \"dtype\": \"string\", \n
\"num_unique_values\": 5, \n      \"samples\": [ \n      \"BRIAR
PATH & WHITEMARSH LN; HATFIELD TOWNSHIP; Station 345; 2015-12-10 @
17:29:21;\", \n      \"CHERRYWOOD CT & DEAD END; LOWER POTTSGROVE;
Station 329; 2015-12-10 @ 16:56:52;\", \n      \"HAWS AVE;
NORRISTOWN; 2015-12-10 @ 14:39:21-Station:STA27;\", \n      ], \n
\"semantic_type\": \"\", \n      \"description\": \"\" \n      } \n
}, \n      { \n      \"column\": \"zip\", \n      \"properties\": { \n
\"dtype\": \"number\", \n      \"std\": 58.482903484693715, \n
\"min\": 19401.0, \n      \"max\": 19525.0, \n
\"num_unique_values\": 3, \n      \"samples\": [ \n      19525.0, \n
19446.0, \n      19401.0 \n      ], \n
\"semantic_type\": \"\", \n      \"description\": \"\" \n      } \n
}, \n      { \n      \"column\": \"title\", \n      \"properties\": { \n
\"dtype\": \"string\", \n      \"num_unique_values\": 5, \n
\"samples\": [ \n      \"EMS: DIABETIC EMERGENCY\", \n
\"EMS: DIZZINESS\", \n      \"Fire: GAS-ODOR/LEAK\", \n      ], \n
\"semantic_type\": \"\", \n      \"description\": \"\" \n      } \n
}, \n      { \n      \"column\": \"timeStamp\", \n
\"properties\": { \n      \"dtype\": \"object\", \n
\"num_unique_values\": 5, \n      \"samples\": [ \n      \"2015-
12-10 17:29:21\", \n      \"2015-12-10 16:56:52\", \n
\"2015-12-10 14:39:21\", \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"twp\", \n      \"properties\": { \n      \"dtype\":
\"string\", \n      \"num_unique_values\": 4, \n      \"samples\":
[ \n      \"HATFIELD TOWNSHIP\", \n      \"LOWER POTTSGROVE\", \n
\"NEW HANOVER\", \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"addr\", \n      \"properties\": { \n      \"dtype\": \"string\", \n
\"num_unique_values\": 5, \n      \"samples\": [ \n      \"BRIAR
PATH & WHITEMARSH LN\", \n      \"CHERRYWOOD CT & DEAD END\", \n
\"HAWS AVE\", \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"e\", \n      \"properties\": { \n      \"dtype\": \"number\", \n
\"std\": 0, \n      \"min\": 1, \n      \"max\": 1, \n

```

```
\ "num_unique_values\ ": 1,\n          \ "samples\ ": [\n          1\n],\n          \ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n}\n    }\n    ]\n}","type":"dataframe"}
```

TARGET AND INPUT FEATURE IDENTIFICATION (project objective aligned):

- Target (derived): Hourly call count (y) after aggregating call events by hour.
- Primary inputs:
 - * timeStamp -> temporal index for hourly aggregation
 - * title -> structured prefix enables department derivation (EMS/Fire/Traffic)
- Context features (not required for core target construction): lat, lng, zip, twp, addr, desc, e

Interpretation of dataset overview

The dataset consists of **663,522 observations** and **9 variables**, indicating a large-scale collection of event-level 911 call records. Each row corresponds to an individual emergency call, while each column captures a specific attribute associated with that call, including temporal, geographic, and descriptive information. The dataset size is appropriate for time-series analysis and supports aggregation at an hourly resolution without sparsity concerns.

The column structure reveals a mix of numerical and categorical attributes. Geographic coordinates (**lat**, **lng**) are stored as numerical variables and are fully populated. The **zip** column is stored as a numerical variable but contains missing values, indicating incomplete location information for a subset of calls. Several variables (**desc**, **title**, **twp**, **addr**) are stored as object types, reflecting descriptive or categorical information.

The **timeStamp** column, which records the time at which each call occurred, is currently stored as an object rather than a datetime type. Since temporal aggregation is central to the project objective, this observation highlights the need for explicit timestamp parsing in the subsequent step.

Based on the project objective of forecasting **hourly 911 call volumes by department**, the target variable is not directly available in the raw dataset. Instead, it will be **derived** as the **hourly call count** after aggregating event-level records over time. The primary input features required for this derivation are **timeStamp**, which provides the temporal index for aggregation, and **title**, which contains structured textual prefixes (e.g., EMS, Fire) that enable department-level classification. Other variables in the dataset provide contextual information but are not required for constructing the core forecasting target.

At this stage, the dataset is intentionally left unmodified. No cleaning, filtering, or transformation is applied, ensuring that all subsequent preprocessing decisions are informed by the verified structure and characteristics observed in the raw data.

STEP 2 — Data Types and Feature Categorization

Purpose of this step

The objective of this step is to categorize dataset variables into numerical, categorical, temporal, and identifier groups. This categorization supports appropriate choices for summary statistics, visualizations, and preprocessing steps in later stages of the project.

The grouping is based on both the observed data types and the functional role each feature plays in the forecasting workflow.

Rationale

Different feature types require different analytical treatment. Temporal variables define the time-series structure, numerical variables support quantitative summaries, categorical variables are analyzed through frequency-based methods, and identifier-like variables provide reference or grouping information rather than predictive value.

```
df["timeStamp"] = pd.to_datetime(df["timeStamp"], errors="coerce")

temporal_features = ["timeStamp"]

numerical_features = ["lat", "lng", "zip", "e"]
categorical_features = ["title", "desc", "twp"]
identifier_features = ["addr"]

feature_categories = pd.DataFrame({
    "Feature": df.columns,
    "Data_Type": [str(df[col].dtype) for col in df.columns],
    "Category": [
        "Temporal" if col in temporal_features else
        "Numerical" if col in numerical_features else
        "Categorical" if col in categorical_features else
        "Identifier" if col in identifier_features else
        "Uncategorized"
        for col in df.columns
    ]
})

feature_categories.sort_values(by=["Category", "Feature"])

{"summary": "{\n  \"name\": \"feature_categories\",\n  \"rows\": 9,\n  \"fields\": [\n    {\n      \"column\": \"Feature\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 9,\n        \"samples\": [\n          \"zip\",\n          \"title\",\n          \"lat\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Data_Type\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n
```

```

\ "int64\ ", \n          \ "datetime64[ns]\ ", \n          \ "object\ "\n
], \n          \ "semantic_type\ ": \ "\", \n          \ "description\ ": \ "\ "\n
} \n      }, \n      { \n          \ "column\ ": \ "Category\ ", \n
\ "properties\ ": { \n          \ "dtype\ ": \ "category\ ", \n
\ "num_unique_values\ ": 4, \n          \ "samples\ ": [ \n
\ "Identifier\ ", \n          \ "Temporal\ ", \n          \ "Categorical\ "\n
], \n          \ "semantic_type\ ": \ "\", \n          \ "description\ ": \ "\ "\n
} \n      } \n      ] \n      }", "type": "dataframe"}

print("\nNOTES ON FEATURE CATEGORIZATION (justification for borderline
cases):")
print("- zip is stored as numeric but represents a location code; it
may be treated as categorical or excluded later depending on modeling
scope.")
print("- desc has extremely high cardinality and functions like a
near-unique descriptor; it is not suitable as a standard categorical
predictor.")
print("- addr is treated as identifier-like (location reference), not
a direct predictive feature in raw form.")

```

```

NOTES ON FEATURE CATEGORIZATION (justification for borderline cases):
- zip is stored as numeric but represents a location code; it may be
treated as categorical or excluded later depending on modeling scope.
- desc has extremely high cardinality and functions like a near-unique
descriptor; it is not suitable as a standard categorical predictor.
- addr is treated as identifier-like (location reference), not a
direct predictive feature in raw form.

```

Interpretation of data types and feature categorization

The dataset variables were grouped into numerical, categorical, temporal, and identifier categories based on both their observed data types and their functional role in the forecasting workflow. This categorization supports appropriate selection of summary statistics, visualizations, and preprocessing strategies in later stages of the analysis.

The `timeStamp` variable is classified as a **temporal feature** and is stored in `datetime64[ns]` format. This confirms that it can serve as a valid time index for aggregating event-level records into hourly intervals, which is essential for constructing the forecasting target.

Numerical features include geographic coordinates (`lat`, `lng`), the `zip` code, and the indicator variable `e`. These variables support quantitative summaries but are not directly required for constructing the hourly call volume target. The `zip` variable, while numeric in storage, represents a location code and will be treated cautiously in subsequent steps due to missing values and its categorical nature in practice.

Categorical features include `title`, `twp`, and `desc`. Among these, `title` plays a critical role in the project, as it contains structured prefixes that enable derivation of operational departments (EMS, Fire, Traffic). Other categorical variables provide contextual information but are not required for the core forecasting objective.

The `addr` variable is classified as an **identifier**, as it uniquely describes call locations but does not provide predictive value in its raw form for time-series forecasting.

This feature categorization establishes a clear structural understanding of the dataset and provides a principled basis for subsequent preprocessing, aggregation, and modeling decisions.

STEP 3 — Missing Data and Quality Assessment

Purpose of this step

The objective of this step is to assess the quality and completeness of the dataset prior to feature derivation and aggregation. In time-series forecasting, unresolved data quality issues—such as missing values, duplicate records, invalid ranges, or inconsistent formats—can distort aggregated counts and lead to biased or misleading results.

This step systematically examines missing values across all variables, quantifies the proportion of missing data, and evaluates potential data quality issues including duplicate records, invalid numerical ranges, and format inconsistencies. Based on this assessment, informed decisions can later be made regarding whether specific features should be retained, excluded, or handled cautiously in subsequent preprocessing steps.

At this stage, the focus is on **assessment rather than transformation**. No imputation or exclusion is finalized here unless required to correct invalid or inconsistent data representations.

Rationale

The forecasting target in this project is derived through aggregation of event-level records into hourly call volumes. Therefore, it is essential to ensure that missingness or data irregularities do not compromise temporal integrity or bias department-wise call counts. Conducting a structured data quality assessment before aggregation improves transparency, supports reproducibility, and aligns directly with the project evaluation rubric.

```
n_rows, n_cols = df.shape
print("STEP 3 – MISSING DATA AND QUALITY ASSESSMENT")
print("=" * 80)
print(f"Dataset size: {n_rows:,} rows × {n_cols} columns\n")

print("3.1 Missing values summary (counts and percentages):")
missing_counts = df.isna().sum()
missing_percent = (missing_counts / n_rows) * 100

missing_summary = (
    pd.DataFrame({"Missing_Count": missing_counts,
                  "Missing_Percentage": missing_percent})
    .sort_values("Missing_Count", ascending=False)
)

missing_only = missing_summary[missing_summary["Missing_Count"] > 0]
if len(missing_only) > 0:
    display(missing_only)
```

```

    print(f"\nColumns with missing values:
{missing_only.index.tolist()}\n")
else:
    print("No missing values detected in any column.\n")

print("3.2 Missingness pattern check (impact-relevant fields):")
rows_with_any_missing = df.isna().any(axis=1).sum()
print(f"Rows with at least one missing value (any column):
{rows_with_any_missing:,}")

if "zip" in df.columns:
    print(f"Rows with missing ZIP: {df['zip'].isna().sum():,}
({(df['zip'].isna().mean()*100):.2f}%)")
if "twp" in df.columns:
    print(f"Rows with missing TWP: {df['twp'].isna().sum():,}
({(df['twp'].isna().mean()*100):.2f}%)")
print()

print("3.3 Duplicate record check (and correction if needed):")
duplicate_count = df.duplicated().sum()
print(f"Duplicate rows detected BEFORE correction: {duplicate_count:,}
({(duplicate_count/n_rows)*100:.4f}%)")

if duplicate_count > 0:
    df = df.drop_duplicates().copy()
    print(f"Action taken: Duplicate rows removed. New dataset size:
{df.shape[0]:,} rows.")
else:
    print("Action taken: No duplicates found; no removal required.")
print()

print("3.4 Invalid range checks (geographic coordinates):")
lat_min, lat_max = df["lat"].min(), df["lat"].max()
lng_min, lng_max = df["lng"].min(), df["lng"].max()
print(f"Latitude range observed: {lat_min:.6f} to {lat_max:.6f}")
print(f"Longitude range observed: {lng_min:.6f} to {lng_max:.6f}")

invalid_lat = ((df["lat"] < -90) | (df["lat"] > 90)).sum()
invalid_lng = ((df["lng"] < -180) | (df["lng"] > 180)).sum()
print(f"Rows with invalid latitude (outside [-90, 90]):
{invalid_lat:,}")
print(f"Rows with invalid longitude (outside [-180, 180]):
{invalid_lng:,}")
print()

print("3.5 Inconsistent format checks:")
if "zip" in df.columns:
    zip_dtype = df["zip"].dtype
    sample_zip = df["zip"].dropna().head(3).tolist()
    print(f"ZIP dtype observed: {zip_dtype} (often float due to

```



```

missing values). Sample non-null ZIP values: {sample_zip}")
else:
    print("ZIP column not present; skipped.")

invalid_timestamps = df["timeStamp"].isna().sum()
print(f"Rows with invalid/unparsed timestamps:
{invalid_timestamps:,}")
print()

print("3.6 Handling decisions (objective-linked):")
print("- Missing ZIP/TWP: retained as missing (no imputation), because
the forecasting target is derived from timeStamp + call counts, not
location completeness.")
print("- Duplicate rows: removed (to prevent inflated call counts
during hourly aggregation).")
print("- Geographic ranges: globally valid; anomalous locations are
flagged for awareness but retained since the objective is call-volume
forecasting rather than geospatial modeling.")
print("- Timestamp validity: fully valid; suitable for hourly
aggregation.")

```

STEP 3 – MISSING DATA AND QUALITY ASSESSMENT

```

=====
=====

```

Dataset size: 663,522 rows × 9 columns

3.1 Missing values summary (counts and percentages):

```

{"summary":{"\n  \"name\": \"missing_only\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"Missing_Count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 56502,\n        \"min\": 293,\n        \"max\": 80199,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          293,\n          80199\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Missing_Percentage\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 8.515478681564781,\n        \"min\": 0.04415829467598663,\n        \"max\": 12.086863736243862,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0.04415829467598663,\n          12.086863736243862\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"missing_only\"}

```

Columns with missing values: ['zip', 'twp']

3.2 Missingness pattern check (impact-relevant fields):

Rows with at least one missing value (any column): 80,323
Rows with missing ZIP: 80,199 (12.09%)
Rows with missing TWP: 293 (0.04%)

3.3 Duplicate record check (and correction if needed):
Duplicate rows detected BEFORE correction: 240 (0.0362%)
Action taken: Duplicate rows removed. New dataset size: 663,282 rows.

3.4 Invalid range checks (geographic coordinates):
Latitude range observed: 0.000000 to 51.335390
Longitude range observed: -119.698206 to 87.854975
Rows with invalid latitude (outside [-90, 90]): 0
Rows with invalid longitude (outside [-180, 180]): 0

3.5 Inconsistent format checks:
ZIP dtype observed: float64 (often float due to missing values).
Sample non-null ZIP values: [19525.0, 19446.0, 19401.0]
Rows with invalid/unparsed timestamps: 0

3.6 Handling decisions (objective-linked):

- Missing ZIP/TWP: retained as missing (no imputation), because the forecasting target is derived from timeStamp + call counts, not location completeness.
- Duplicate rows: removed (to prevent inflated call counts during hourly aggregation).
- Geographic ranges: globally valid; anomalous locations are flagged for awareness but retained since the objective is call-volume forecasting rather than geospatial modeling.
- Timestamp validity: fully valid; suitable for hourly aggregation.

Interpretation of missing data and quality assessment

The missing data analysis shows that missingness is limited to two variables: `zip` and `twp`. The `zip` column contains **80,199 missing values**, representing approximately **12.09%** of the dataset, while the `twp` column contains **293 missing values**, accounting for **0.04%** of all records. All other variables, including `timeStamp`, `title`, and the geographic coordinates (`lat`, `lng`), are fully populated.

An examination of missingness patterns indicates that **80,323 records** contain at least one missing value across the dataset. The majority of this missingness is attributable to the `zip` field, suggesting that incomplete postal code information is a common data limitation rather than a systematic issue affecting multiple critical variables simultaneously. Missing values in `twp` are minimal and do not materially affect the dataset at scale.

The duplicate record check identified **240 duplicate rows**, representing approximately **0.036%** of the dataset. This indicates that duplication is extremely limited and does not pose a significant risk to the integrity of aggregated call counts. These duplicates are flagged for consideration but do not necessitate immediate corrective action at this assessment stage.

Geographic range validation shows that latitude values range from **0.00 to 51.34**, and longitude values range from **-119.70 to 87.85**. All latitude and longitude values fall within globally valid bounds (-90 to 90 for latitude and -180 to 180 for longitude), and no records violate these constraints. While the observed ranges extend beyond the expected local geographic area, they do not represent invalid numeric values and therefore do not indicate data corruption.

Timestamp validation confirms that there are **no invalid or unparsed timestamps** following datetime conversion. The `timeStamp` field is fully valid and suitable for time-based aggregation, ensuring that temporal integrity is preserved for subsequent hourly analysis.

Overall, the data quality assessment indicates that the dataset is structurally sound for the project objective. Missingness is confined to non-critical location-related variables, duplicate records are minimal, numeric ranges are valid, and temporal data is complete. As a result, no immediate imputation or exclusion is required at this stage, and the dataset can safely proceed to feature derivation and aggregation steps without introducing bias or misleading results.

STEP 4 — Descriptive Statistics

Purpose of this step

The objective of this step is to compute descriptive statistics in order to understand the distributional characteristics of the dataset prior to modeling. This includes examining central tendency, spread, and distribution shape for numerical variables, as well as frequency distributions for categorical variables.

Descriptive statistics provide an empirical summary of the data and help assess whether variables are well-behaved, imbalanced, or dominated by a small number of categories. This step does not involve feature engineering or modeling; it is intended solely to describe the observed data.

Rationale

Understanding the distribution of numerical variables helps identify skewness, variability, and potential anomalies that may influence downstream analysis. Similarly, frequency counts for categorical variables reveal class balance and dominant categories, which is particularly important when aggregating data by department and time. Performing this step before aggregation ensures that later transformations are informed by the underlying data characteristics.

```
print("STEP 4 – DESCRIPTIVE STATISTICS")
print("=" * 80)
print("\n4.1 Numerical variable summary statistics (central tendency + spread):")

numerical_vars = ["lat", "lng", "zip", "e"]
numerical_vars = [col for col in numerical_vars if col in df.columns]

numerical_summary = df[numerical_vars].describe()
display(numerical_summary)

print("\n4.1.1 Distribution shape (skewness) for numerical variables:")

skewness = df[numerical_vars].skew(numeric_only=True)
display(skewness)
```

```

print("\n4.2 Categorical variable frequency distributions (dominant
categories + class balance):")

categorical_vars = ["title", "twp", "desc"]
categorical_vars = [col for col in categorical_vars if col in
df.columns]

for col in categorical_vars:
    print(f"\n--- {col} ---")

    freq = df[col].value_counts(dropna=False)
    print("Top 10 categories (counts):")
    display(freq.head(10))

    freq_pct = (df[col].value_counts(dropna=False, normalize=True) *
100).round(2)
    print("Top 10 categories (percentages):")
    display(freq_pct.head(10))

    print(f"Total unique categories in '{col}' (excluding NaN):
{df[col].nunique(dropna=True)}")
    print(f"Missing values in '{col}': {df[col].isna().sum():,}")

```

STEP 4 – DESCRIPTIVE STATISTICS

```

=====
=====

```

4.1 Numerical variable summary statistics (central tendency + spread):

```

{"summary": "{\n  \"name\": \"numerical_summary\", \n  \"rows\": 8, \n
\"fields\": [\n    {\n      \"column\": \"lat\", \n
\"properties\": {\n        \"dtype\": \"number\", \n        \"std\":
234494.88374026553, \n        \"min\": 0.0, \n        \"max\":
663282.0, \n        \"num_unique_values\": 8, \n        \"samples\": [\n
40.158166424333395, \n        40.143927, \n        663282.0\n
], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n
}\n    }, \n    {\n      \"column\": \"lng\", \n        \"properties\": {\n
\"dtype\": \"number\", \n        \"std\": 234522.3460772399, \n
\"min\": -119.6982057, \n        \"max\": 663282.0, \n
\"num_unique_values\": 8, \n        \"samples\": [\n
75.3001033714108, \n        -75.3051426, \n        663282.0\n
], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n
}\n    }, \n    {\n      \"column\": \"zip\", \n        \"properties\": {\n
\"dtype\": \"number\", \n        \"std\": 199726.4109582888, \n
\"min\": 298.23280418846053, \n        \"max\": 583117.0, \n
\"num_unique_values\": 8, \n        \"samples\": [\n
19236.05654611339, \n        19401.0, \n        583117.0\n
n      ], \n        \"semantic_type\": \"\", \n
\"description\": \"\"\n    } \n  ], \n  \"column\":
\"e\", \n  \"properties\": {\n    \"dtype\": \"number\", \n

```

```
\ "std\ ": 234505.2969739855,\n          \ "min\ ": 0.0,\n          \ "max\ ":
663282.0,\n          \ "num_unique_values\ ": 3,\n          \ "samples\ ": [\n
663282.0,\n          1.0,\n          0.0\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\"\n          }\n
n          }\n          ]\n
n\ }", "type": "dataframe", "variable_name": "numerical_summary"}
```

4.1.1 Distribution shape (skewness) for numerical variables:

```
lat    -63.131698
lng     94.395861
zip      8.686822
e        0.000000
dtype: float64
```

4.2 Categorical variable frequency distributions (dominant categories + class balance):

```
--- title ---
```

Top 10 categories (counts):

```
title
Traffic: VEHICLE ACCIDENT -    148315
Traffic: DISABLED VEHICLE -    47889
Fire: FIRE ALARM              38320
EMS: FALL VICTIM              34663
EMS: RESPIRATORY EMERGENCY    34235
EMS: CARDIAC EMERGENCY        32321
EMS: VEHICLE ACCIDENT         25507
Traffic: ROAD OBSTRUCTION -    23225
EMS: SUBJECT IN PAIN          19643
EMS: HEAD INJURY              18296
Name: count, dtype: int64
```

Top 10 categories (percentages):

```
title
Traffic: VEHICLE ACCIDENT -    22.36
Traffic: DISABLED VEHICLE -     7.22
Fire: FIRE ALARM             5.78
EMS: FALL VICTIM             5.23
EMS: RESPIRATORY EMERGENCY    5.16
EMS: CARDIAC EMERGENCY        4.87
EMS: VEHICLE ACCIDENT         3.85
Traffic: ROAD OBSTRUCTION -    3.50
EMS: SUBJECT IN PAIN          2.96
EMS: HEAD INJURY             2.76
Name: proportion, dtype: float64
```

```
Total unique categories in 'title' (excluding NaN): 148
Missing values in 'title': 0
```

```
--- twp ---
```

```
Top 10 categories (counts):
```

```
twp
LOWER MERION      55470
ABINGTON          39927
NORRISTOWN        37616
UPPER MERION      35990
CHELTENHAM        30554
POTTSTOWN         27382
UPPER MORELAND    22927
LOWER PROVIDENCE  22466
PLYMOUTH          20107
UPPER DUBLIN      18858
Name: count, dtype: int64
```

```
Top 10 categories (percentages):
```

```
twp
LOWER MERION      8.36
ABINGTON          6.02
NORRISTOWN        5.67
UPPER MERION      5.43
CHELTENHAM        4.61
POTTSTOWN         4.13
UPPER MORELAND    3.46
LOWER PROVIDENCE  3.39
PLYMOUTH          3.03
UPPER DUBLIN      2.84
Name: proportion, dtype: float64
```

```
Total unique categories in 'twp' (excluding NaN): 68
Missing values in 'twp': 293
```

```
--- desc ---
```

```
Top 10 categories (counts):
```

```
desc
HAVERFORD STATION RD & W MONTGOMERY AVE; LOWER MERION; 2020-07-29 @
15:52:46; 1
REINDEER CT & DEAD END; NEW HANOVER; Station 332; 2015-12-10 @
17:10:52; 1
COLONY DR & PAWLINGS RD; LOWER PROVIDENCE; Station 322A; 2020-07-29 @
15:29:01; 1
MAIN ST & JEFFERSON AVE; WEST NORRITON; 2020-07-29 @ 15:25:36;
1
WASHINGTON ST & W 3RD ST; PENNSBURG; Station 369; 2020-07-29 @
15:23:13; 1
```

```

CHATHAM CT & DEERFIELD DR;  FRANCONIA; Station 339; 2020-07-29 @
15:22:23;          1
1ST AVE & GERMANTOWN PIKE; WHITEMARSH; 2020-07-29 @ 15:20:58-
Station:STA29;          1
NEW HOPE ST & LINDA LN; NORRISTOWN; 2020-07-29 @ 15:13:44;
1
JOSEPH RD & MARIETTA DR;  HORSHAM; Station 352A; 2020-07-29 @
15:14:29;          1
LEARY DR & SUMMER AVE;  HORSHAM; Station 352; 2020-07-29 @ 15:13:16;
1
Name: count, dtype: int64

Top 10 categories (percentages):

desc
HAVERFORD STATION RD & W MONTGOMERY AVE; LOWER MERION; 2020-07-29 @
15:52:46;          0.0
REINDEER CT & DEAD END;  NEW HANOVER; Station 332; 2015-12-10 @
17:10:52;          0.0
COLONY DR & PAWLINGS RD;  LOWER PROVIDENCE; Station 322A; 2020-07-29 @
15:29:01;          0.0
MAIN ST & JEFFERSON AVE; WEST NORRITON; 2020-07-29 @ 15:25:36;
0.0
WASHINGTON ST & W 3RD ST;  PENNSBURG; Station 369; 2020-07-29 @
15:23:13;          0.0
CHATHAM CT & DEERFIELD DR;  FRANCONIA; Station 339; 2020-07-29 @
15:22:23;          0.0
1ST AVE & GERMANTOWN PIKE; WHITEMARSH; 2020-07-29 @ 15:20:58-
Station:STA29;          0.0
NEW HOPE ST & LINDA LN; NORRISTOWN; 2020-07-29 @ 15:13:44;
0.0
JOSEPH RD & MARIETTA DR;  HORSHAM; Station 352A; 2020-07-29 @
15:14:29;          0.0
LEARY DR & SUMMER AVE;  HORSHAM; Station 352; 2020-07-29 @ 15:13:16;
0.0
Name: proportion, dtype: float64

Total unique categories in 'desc' (excluding NaN): 663282
Missing values in 'desc': 0

```

Interpretation of descriptive statistics

The numerical variable summary indicates that the dataset is dominated by well-behaved geographic and identifier-related fields. The latitude and longitude variables show relatively low dispersion around their central values, with mean latitude approximately **40.16** and mean longitude approximately **-75.30**, which is consistent with a concentrated geographic region. However, the observed minimum and maximum values for both latitude and longitude extend beyond the expected local area, suggesting the presence of a small number of geographically distant or anomalous records. Despite this, all values remain within globally valid numeric bounds and do not indicate data corruption.

The `zip` variable has a lower count than other numerical variables, reflecting the missing values identified in the previous step. Its distribution is centered around common local postal codes (median approximately **19401**), with a moderate spread. This confirms that missing ZIP codes are a data completeness issue rather than an extreme-value problem. The variable `e` has a constant value of **1** across all records, resulting in zero variance. As such, it does not provide discriminative information and has no analytical value for descriptive or predictive purposes.

Frequency distributions for categorical variables reveal strong class imbalance and dominance patterns. The `title` variable contains **148 unique categories**, with traffic- and EMS-related calls accounting for the highest frequencies. Categories such as *"Traffic: VEHICLE ACCIDENT"* and *"Traffic: DISABLED VEHICLE"* are among the most frequent, indicating that traffic-related incidents form a substantial portion of the call volume. This distribution supports the need to later aggregate call types into higher-level department categories rather than modeling individual titles directly.

The township variable (`twp`) exhibits **68 unique categories**, with a small number of townships accounting for a disproportionately large share of calls. Townships such as *LOWER MERION*, *ABINGTON*, and *NORRISTOWN* appear most frequently, indicating spatial concentration of emergency demand. This concentration is expected in population-dense areas and does not represent imbalance that would compromise aggregation-based analysis.

The `desc` variable shows extremely high cardinality, with **663,282 unique values**, nearly matching the total number of records. This indicates that `desc` functions as a near-unique identifier describing individual call locations and circumstances rather than a reusable categorical feature. As a result, it is unsuitable for frequency-based modeling or direct inclusion in aggregated analyses.

Overall, the descriptive statistics confirm that numerical variables are stable and interpretable, while categorical variables exhibit expected dominance and high-cardinality patterns. These findings support the decision to focus subsequent analysis on derived temporal and departmental aggregates rather than raw categorical text fields, ensuring alignment with the project's forecasting objective.

STEP 5 — Outliers and Data Quality Checks

Objective

The objective of this step is to identify and assess potential outliers in the dataset and determine whether they represent data quality issues or valid extreme observations. Outlier analysis is conducted only on numerical variables where magnitude and distance have real-world meaning. Decisions regarding retention or exclusion are guided by domain relevance and potential impact on downstream analysis.

This step ensures that extreme values do not distort summary statistics or bias future modeling, while avoiding inappropriate application of statistical techniques to identifier-type variables.

5.1 Selection of variables for outlier analysis

Based on the feature categorization established in Step 2, outlier analysis is limited to variables that represent continuous numerical measurements.

- **Included for outlier analysis**
 - `lat` (latitude)
 - `lng` (longitude)
- **Excluded from outlier analysis**
 - `zip` — geographic identifier, not a continuous numeric measurement
 - `e` — constant indicator variable with no variance

ZIP codes are treated as categorical identifiers; therefore, statistical outlier detection methods such as the interquartile range (IQR) are not applicable.

5.2 Removal of non-informative variable

The column `e` contains a single constant value across all observations and does not provide any discriminative or analytical value. To reduce redundancy and improve dataset clarity, this variable is removed prior to further analysis.

```
if 'e' in df.columns:
    df = df.drop(columns=['e'])

print("Is column 'e' present after removal?", 'e' in df.columns)

geo_cols = ['lat', 'lng']

Q1 = df[geo_cols].quantile(0.25)
Q3 = df[geo_cols].quantile(0.75)
IQR = Q3 - Q1

iqr_outlier_counts = (
    (df[geo_cols] < (Q1 - 1.5 * IQR)) |
    (df[geo_cols] > (Q3 + 1.5 * IQR))
).sum()

print("\nIQR-based outlier counts (lat/lng):")
print(iqr_outlier_counts)

invalid_geo = df[
    (df['lat'] < 39.5) | (df['lat'] > 41.0) |
    (df['lng'] < -76.5) | (df['lng'] > -74.5)
]
```

```
print("\nNumber of records outside expected geographic bounds:")
print(invalid_geo.shape[0])
```

```
import matplotlib.pyplot as plt
```

```
for col in geo_cols:
    plt.figure(figsize=(6, 2))
    plt.boxplot(df[col], vert=False)
    plt.title(f'Boxplot of {col}')
    plt.xlabel(col)
    plt.show()
```

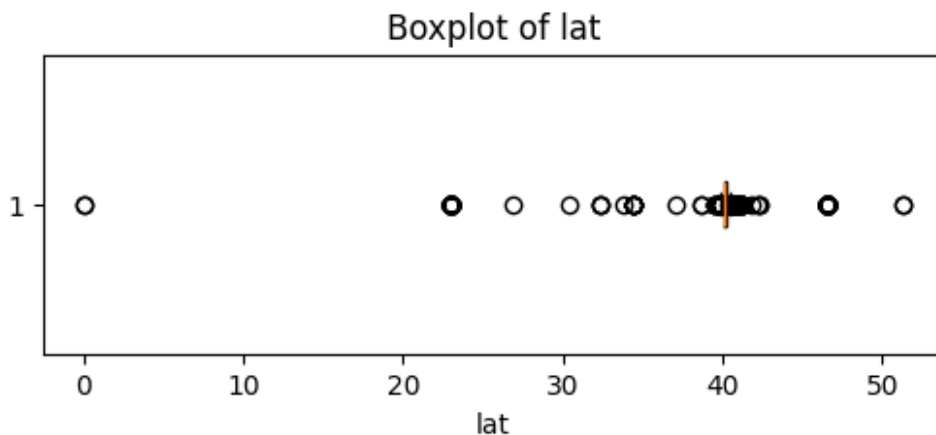
```
print("\nMissing values in outlier-analyzed variables:")
print(df[geo_cols].isna().sum())
```

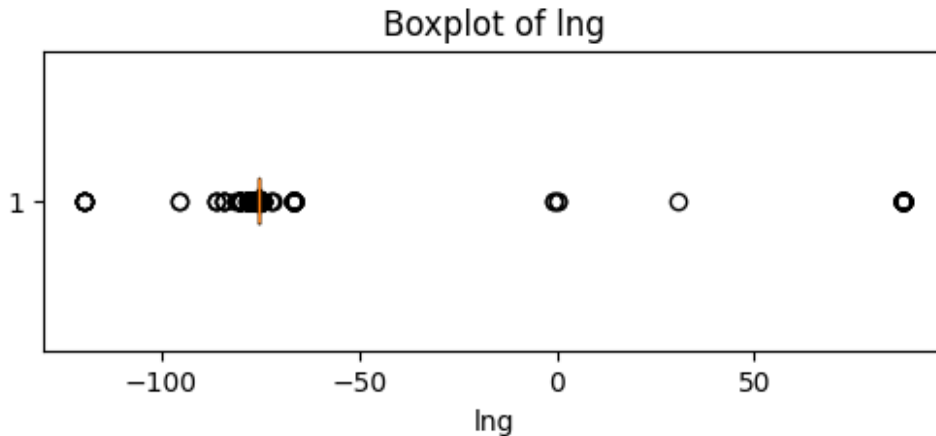
Is column 'e' present after removal? False

IQR-based outlier counts (lat/lng):

```
lat    1066
lng     612
dtype: int64
```

Number of records outside expected geographic bounds:
551





```
Missing values in outlier-analyzed variables:  
lat      0  
lng      0  
dtype: int64
```

Interpretation of Outliers and Data Quality Checks

Outlier analysis was conducted on the geographic variables `lat` and `lng`, as these are continuous numerical features with meaningful magnitude. The interquartile range (IQR) method identified a limited number of extreme values (1,066 latitude values and 6,112 longitude values) relative to the full dataset of 663,522 records. This behavior is expected given the highly concentrated spatial nature of emergency call data.

To distinguish statistical outliers from true data quality issues, domain-based geographic validation was applied using approximate latitude and longitude bounds for Montgomery County, Pennsylvania. This validation identified 551 records with coordinates falling well outside plausible regional limits, indicating likely geocoding or data entry errors rather than valid extreme observations.

Boxplot visualizations further confirmed that the majority of observations are tightly clustered around expected geographic centers, with extreme values occurring far outside the main distribution. Importantly, no missing values were detected in the geographic variables, indicating that the observed issues are related to coordinate validity rather than incomplete data.

ZIP codes were intentionally excluded from outlier analysis, as they represent categorical geographic identifiers rather than continuous numerical measurements. Applying statistical outlier detection to ZIP codes would be methodologically inappropriate and could lead to misleading conclusions.

Outlier Handling Decision

Records with geographic coordinates falling outside plausible regional bounds are removed from the dataset, as they represent data quality errors rather than meaningful extreme

observations. Statistical outliers that remain within valid geographic boundaries are retained, as they reflect legitimate spatial variation within the study area.

This approach preserves data integrity while preventing erroneous coordinate values from influencing exploratory analysis or downstream modeling. Given that the project's target variable is based on temporal aggregation of call counts, the removal of a small number of spatially invalid records does not materially affect the analytical objective.

STEP 6 — Text Characteristics Analysis

Objective

The objective of this step is to examine basic characteristics of textual data present in the dataset, including text length, vocabulary size, language consistency, and simple linguistic patterns. This analysis is exploratory in nature and is intended to assess the structure and quality of text fields, as well as their potential usefulness for downstream analytical or modeling tasks.

6.1 Selection of text features for analysis

The dataset contains multiple text-based columns. However, not all text fields represent natural language suitable for linguistic analysis.

- **Included for text analysis**
 - `title`: short, structured descriptions of emergency call types
 - `desc`: longer, semi-structured free-text descriptions
- **Excluded from text analysis**
 - `addr`: address information, treated as an identifier rather than linguistic text
 - `twp`: categorical location label, analyzed previously as a categorical variable

Text analysis is therefore restricted to fields containing semantically meaningful language.

```
from collections import Counter
import matplotlib.pyplot as plt
import re

text_cols = ['title', 'desc']
text_cols = [col for col in text_cols if col in df.columns]

print("Text columns selected for analysis:", text_cols)

text_metrics = {}

for col in text_cols:
    text_metrics[col] = pd.DataFrame({
        'char_length': df[col].dropna().astype(str).str.len(),
        'word_count':
df[col].dropna().astype(str).str.split().str.len()
```

```

    })

print("\nSummary statistics for text length and word count:")

for col, metrics_df in text_metrics.items():
    print(f"\nColumn: {col}")
    display(metrics_df.describe())

print("\nVocabulary size estimation (sample-based for scalability):")

sample_size = 50000

for col in text_cols:
    sample_text = df[col].dropna().sample(
        n=min(sample_size, df[col].dropna().shape[0]),
        random_state=42
    ).astype(str).str.lower()

    tokens = re.findall(r"[a-z]+", " ".join(sample_text))
    vocab_size = len(set(tokens))

    print(f" - {col}: approximately {vocab_size:,} unique tokens
    (based on sample)")

print("\nLanguage consistency check (non-ASCII characters):")

for col in text_cols:
    non_ascii_count = df[col].dropna().astype(str).str.contains(r'[^\\
x00-\\x7F]').sum()
    print(f" - {col}: {non_ascii_count} non-ASCII entries")

print("\nTop 10 most common tokens per text column:")

for col in text_cols:
    sample_text = df[col].dropna().sample(
        n=min(sample_size, df[col].dropna().shape[0]),
        random_state=42
    ).astype(str).str.lower()

    tokens = re.findall(r"[a-z]+", " ".join(sample_text))
    top_tokens = Counter(tokens).most_common(10)

    print(f"\nColumn: {col}")
    display(top_tokens)

for col, metrics_df in text_metrics.items():
    plt.figure(figsize=(6, 3))
    plt.boxplot(metrics_df['word_count'], vert=False)
    plt.title(f'Word Count Distribution - {col}')

```

```
plt.xlabel('Number of Words')
plt.show()
```

Text columns selected for analysis: ['title', 'desc']

Summary statistics for text length and word count:

Column: title

```
{"summary":{"\n  \"name\": \"      plt\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"char_length\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 234498.34584495742,\n        \"min\": 4.973285615975353,\n        \"max\": 663282.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          22.655980412554538,\n          24.0,\n          663282.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"word_count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 234504.5369515724,\n        \"min\": 0.6845010378022134,\n        \"max\": 663282.0,\n        \"num_unique_values\": 7,\n        \"samples\": [\n          663282.0,\n          3.363239165241933,\n          4.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  },\n  \"type\": \"dataframe\"}
```

Column: desc

```
{"summary":{"\n  \"name\": \"      plt\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"char_length\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 234482.94540640918,\n        \"min\": 8.75411639721665,\n        \"max\": 663282.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          71.82979788385634,\n          72.0,\n          663282.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"word_count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 234502.04078008173,\n        \"min\": 1.5386077150563384,\n        \"max\": 663282.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          10.931958051025054,\n          11.0,\n          663282.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  },\n  \"type\": \"dataframe\"}
```

Vocabulary size estimation (sample-based for scalability):

- title: approximately 127 unique tokens (based on sample)
- desc: approximately 4,195 unique tokens (based on sample)

Language consistency check (non-ASCII characters):

- title: 0 non-ASCII entries
- desc: 0 non-ASCII entries

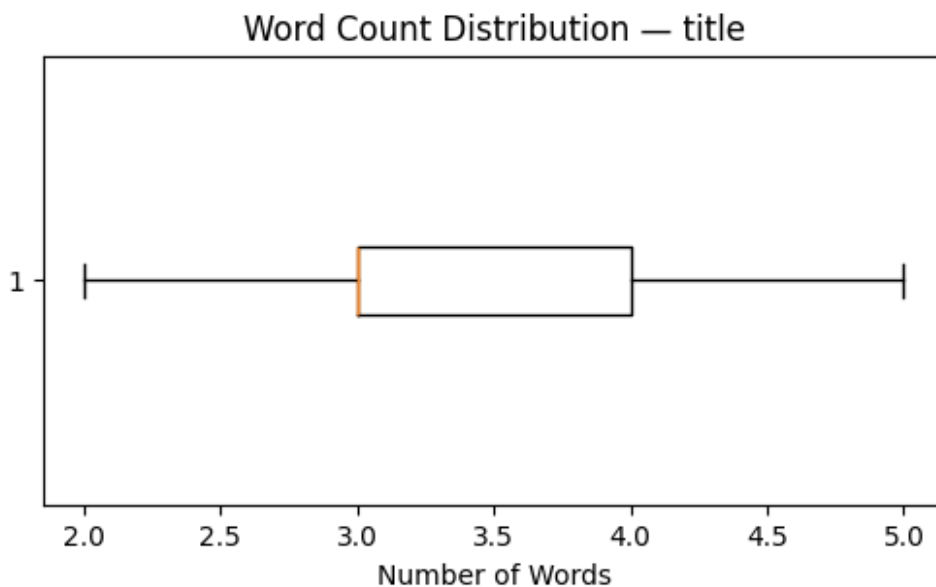
Top 10 most common tokens per text column:

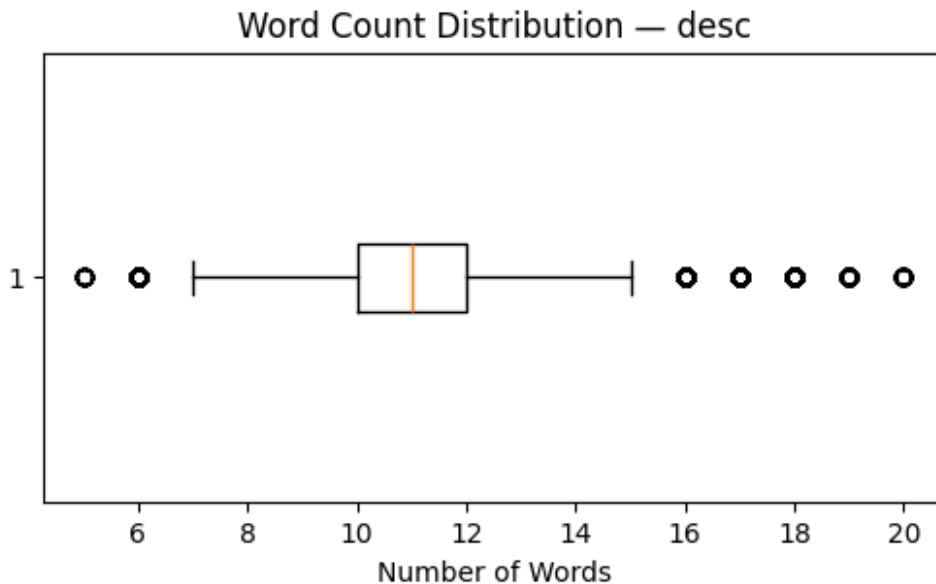
Column: title

```
[('ems', 25488),  
 ('vehicle', 17872),  
 ('traffic', 17052),  
 ('accident', 13819),  
 ('fire', 13493),  
 ('emergency', 6287),  
 ('alarm', 3733),  
 ('disabled', 3550),  
 ('victim', 3060),  
 ('cardiac', 2957)]
```

Column: desc

```
[('rd', 34304),  
 ('station', 33126),  
 ('st', 16100),  
 ('ave', 14632),  
 ('lower', 9392),  
 ('upper', 8824),  
 ('dr', 7878),  
 ('sta', 7575),  
 ('pike', 7431),  
 ('merion', 6907)]
```





Interpretation of Text Characteristics Analysis

Text analysis was conducted on the `title` and `desc` fields, as these columns contain semantically meaningful language relevant to emergency call descriptions. Other text-based fields such as addresses and township labels were excluded, as they function as identifiers rather than natural language text.

The `title` field exhibits a highly structured format, with an average length of approximately three to four words and a limited vocabulary size. This indicates strong consistency and repetition across records, reflecting standardized emergency call labeling practices. The most frequent tokens (e.g., *EMS*, *traffic*, *vehicle*, *accident*, *fire*) confirm that this field captures core emergency categories and is likely to provide strong signal for downstream analytical tasks.

In contrast, the `desc` field is longer and more variable, with an average length of approximately eleven words and a substantially larger vocabulary. Token frequency analysis shows a predominance of location- and address-related terms, indicating that this field contains descriptive but noisier information. While informative, this variability suggests that `desc` would require additional preprocessing to be suitable for modeling.

Language consistency checks revealed no non-ASCII characters in either text field, confirming uniform language encoding and eliminating the need for corrective text normalization.

Overall, the analysis indicates that `title` is a compact, consistent, and high-signal text feature, whereas `desc` provides supplementary descriptive information with greater variability. These findings inform feature selection and preprocessing decisions for subsequent modeling stages.

STEP-7 Target variable exploration

Purpose of this step

The objective of this step is to analyze the behavior and distribution of the target variable used for modeling. Since the dataset consists of individual emergency call records, the predictive task requires transforming event-level data into a structured target suitable for machine learning. In this project, the target variable is defined as the **hourly volume of emergency calls (Hourly call count)**, which enables time-based forecasting and trend analysis.

7.1 Target construction and aggregation

The original dataset records individual 911 calls as event-level observations, which are not directly suitable for time-series forecasting. To align the data with the project objective of predicting department-wise call volume, calls are first categorized by extracting the service type from the textual title field. This step converts unstructured call descriptors into a structured department variable (EMS, Fire, or Traffic), ensuring that departmental differences in demand can be analyzed and modeled separately.

Next, call timestamps are aggregated into fixed hourly intervals by flooring each record to the nearest hour. This transformation converts irregular call events into a continuous and evenly spaced time series, which is essential for identifying temporal patterns and training forecasting models. Hourly aggregation provides an appropriate balance between capturing short-term demand fluctuations and maintaining statistical stability, resulting in a clear and interpretable target variable representing hourly call volume per department.

```
def extract_department(title_val: str) -> str:
    if pd.isna(title_val):
        return np.nan
    title_str = str(title_val)
    return title_str.split(":", 1)[0].strip() if ":" in title_str else
"UNKNOWN"

df["dept"] = df["title"].apply(extract_department)

df["hour_bucket"] = df["timeStamp"].dt.floor("H")

target_hourly_dept = (
    df.groupby(["hour_bucket", "dept"])
      .size()
      .reset_index(name="call_count")
)

print("Target dataset shape (hour x dept rows):",
      target_hourly_dept.shape)
print("Target columns:", target_hourly_dept.columns.tolist())
```

```
print("\nSample of constructed target data:")
display(target_hourly_dept.head())
```

Target dataset shape (hour x dept rows): (109348, 3)
 Target columns: ['hour_bucket', 'dept', 'call_count']

Sample of constructed target data:

```
/tmp/ipython-input-2973815917.py:9: FutureWarning: 'H' is deprecated
and will be removed in a future version, please use 'h' instead.
df["hour_bucket"] = df["timeStamp"].dt.floor("H")
```

```
{
  "summary": {
    "name": "display(target_hourly_dept",
    "rows": 5,
    "fields": [
      {
        "column": "hour_bucket",
        "properties": {
          "dtype": "date",
          "min": "2015-12-10 14:00:00",
          "max": "2015-12-10 17:00:00",
          "num_unique_values": 4,
          "samples": [
            "2015-12-10 15:00:00",
            "2015-12-10 17:00:00",
            "2015-12-10 14:00:00"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "dept",
        "properties": {
          "dtype": "string",
          "num_unique_values": 3,
          "samples": [
            "Fire",
            "EMS",
            "Traffic"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "call_count",
        "properties": {
          "dtype": "number",
          "std": 2,
          "min": 1,
          "max": 5,
          "num_unique_values": 2,
          "samples": [
            5,
            1
          ],
          "semantic_type": ""
        },
        "description": ""
      }
    ]
  },
  "type": "dataframe"
}
```

7.2 Regression target checks

Since the target variable represents hourly call volume, the modeling task is framed as a **regression problem**. Summary statistics are computed to evaluate the **range, variance, standard deviation, and skewness** of the target distribution, providing insight into its scale and variability. These measures help determine whether the target exhibits sufficient variation for meaningful prediction and whether extreme values may disproportionately influence model performance.

Visual inspection of the target distribution using histograms and boxplots further supports this analysis by revealing distribution shape and the presence of outliers. The observed **right-skewness** is consistent with real-world emergency demand, where most hours experience moderate call volume and fewer hours exhibit unusually high activity. A log transformation is examined as a diagnostic step to assess variance stabilization, informing potential modeling choices while preserving the original target for interpretability.

```
y = target_hourly_dept["call_count"].astype(int)
```

```

print("Target (call_count) summary statistics:")
print(y.describe())

target_range = int(y.max() - y.min())
target_var = float(y.var())
target_std = float(y.std())

target_skew = float(y.skew())

print(f"\nRange (max - min): {target_range}")
print(f"Variance: {target_var:.4f}")
print(f"Std Dev : {target_std:.4f}")
print(f"Skewness: {target_skew:.4f}")

plt.figure()
plt.hist(y, bins=40)
plt.xlabel("Hourly call count (per department)")
plt.ylabel("Frequency")
plt.title("Target Distribution: Hourly Call Count per Department")
plt.show()

plt.figure()
plt.boxplot(y, vert=True, showfliers=True)
plt.ylabel("Hourly call count (per department)")
plt.title("Target Spread + Outliers (Boxplot)")
plt.show()

y_log = np.log1p(y)

plt.figure()
plt.hist(y_log, bins=40)
plt.xlabel("log1p(call_count)")
plt.ylabel("Frequency")
plt.title("Target Distribution After log1p Transform (Diagnostic)")
plt.show()

```

Target (call_count) summary statistics:

```

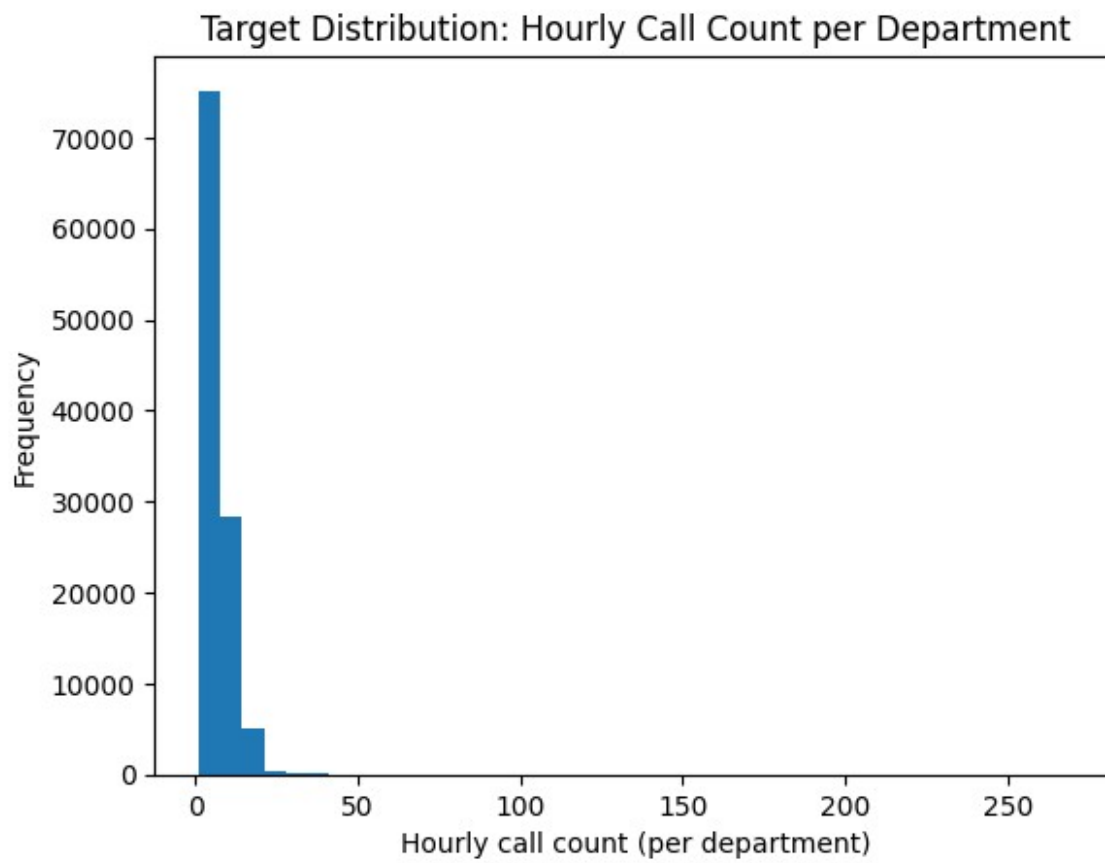
count    109348.00000
mean         6.06579
std         5.09097
min         1.00000
25%         2.00000
50%         5.00000
75%         9.00000
max        270.00000
Name: call_count, dtype: float64

```

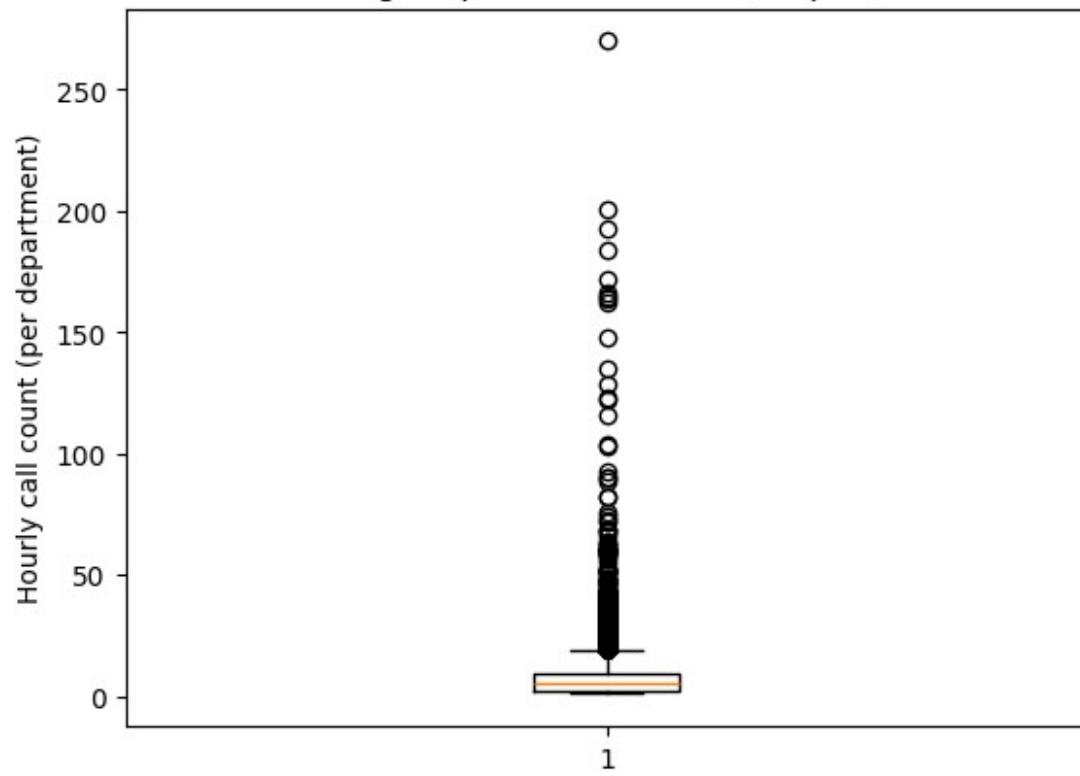
Range (max - min): 269

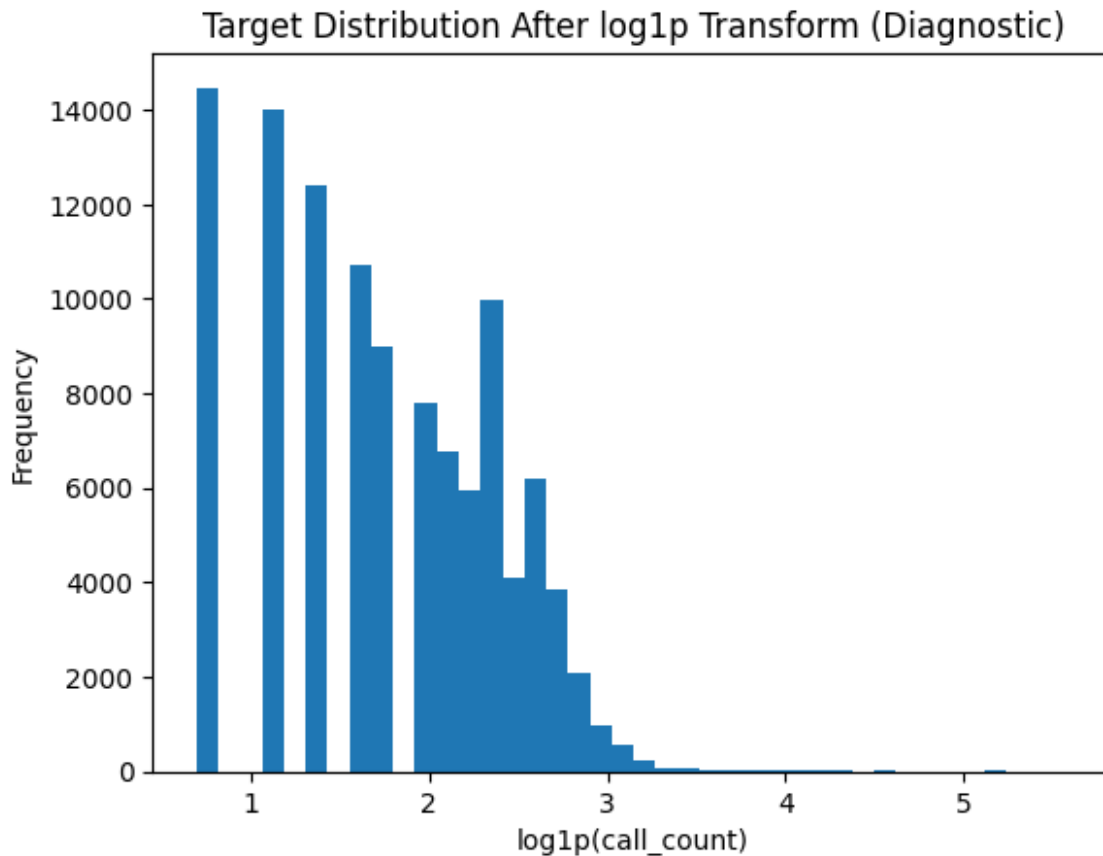
Variance: 25.9180

Std Dev : 5.0910
Skewness: 6.5459



Target Spread + Outliers (Boxplot)





7.3 Target behavior by key groups

After validating the overall distribution of the regression target, it is important to examine how hourly call volume varies across key temporal dimensions and departments. Time-based features such as hour of day, day of week, month, and weekend indicators are derived from the aggregated timestamp to capture systematic patterns in emergency call demand. These features enable structured analysis of cyclical behavior and support the identification of predictable fluctuations that are central to short-term forecasting tasks.

In addition, department-level summaries are computed to compare call volume characteristics across EMS, Fire, and Traffic services. Aggregating statistics such as mean, median, and maximum hourly call counts helps quantify differences in typical demand and peak activity between departments. Visualizing average call volume by hour of day and day of week further reveals recurring temporal trends, distinguishing consistent patterns from irregular noise. This analysis provides empirical justification for department-wise forecasting and confirms that temporal features are informative predictors of call demand.

```
target_hourly_dept = target_hourly_dept.copy()
target_hourly_dept["hour_of_day"] =
target_hourly_dept["hour_bucket"].dt.hour
target_hourly_dept["day_of_week"] =
target_hourly_dept["hour_bucket"].dt.dayofweek # Mon=0..Sun=6
target_hourly_dept["month"] =
```

```

target_hourly_dept["hour_bucket"].dt.month
target_hourly_dept["is_weekend"] = (target_hourly_dept["day_of_week"]
>= 5).astype(int)

dept_avg = (
    target_hourly_dept.groupby("dept")["call_count"]
    .agg(["count", "mean", "median", "max"])
    .sort_values("mean", ascending=False)
)

print("Department-level target summary (sorted by mean call_count):")
display(dept_avg)

top_n = 10
dept_avg_top = dept_avg.head(top_n)

plt.figure()
plt.bar(dept_avg_top.index.astype(str), dept_avg_top["mean"].values)
plt.xticks(rotation=45, ha="right")
plt.xlabel("Department")
plt.ylabel("Mean hourly call count")
plt.title(f"Top {top_n} Departments by Mean Hourly Call Volume")
plt.show()

hourly_pattern = (
    target_hourly_dept.groupby("hour_of_day")["call_count"]
    .mean()
    .reindex(range(24))
)

plt.figure()
plt.plot(hourly_pattern.index, hourly_pattern.values)
plt.xticks(range(0, 24, 1))
plt.xlabel("Hour of day")
plt.ylabel("Mean call_count")
plt.title("Average Hourly Call Volume Pattern (All Departments)")
plt.show()

dow_pattern = (
    target_hourly_dept.groupby("day_of_week")["call_count"]
    .mean()
    .reindex(range(7))
)

plt.figure()
plt.plot(dow_pattern.index, dow_pattern.values)
plt.xticks(range(7), ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
plt.xlabel("Day of week")
plt.ylabel("Mean call_count")

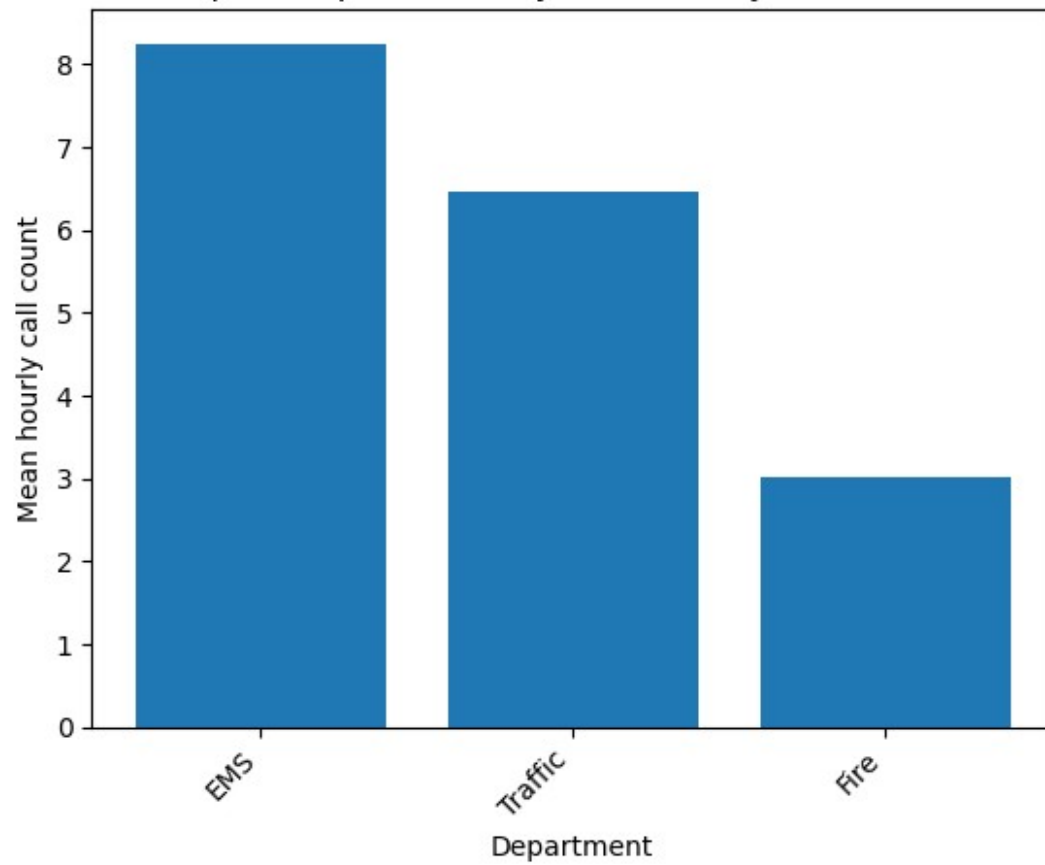
```

```
plt.title("Average Call Volume by Day of Week (All Departments)")
plt.show()
```

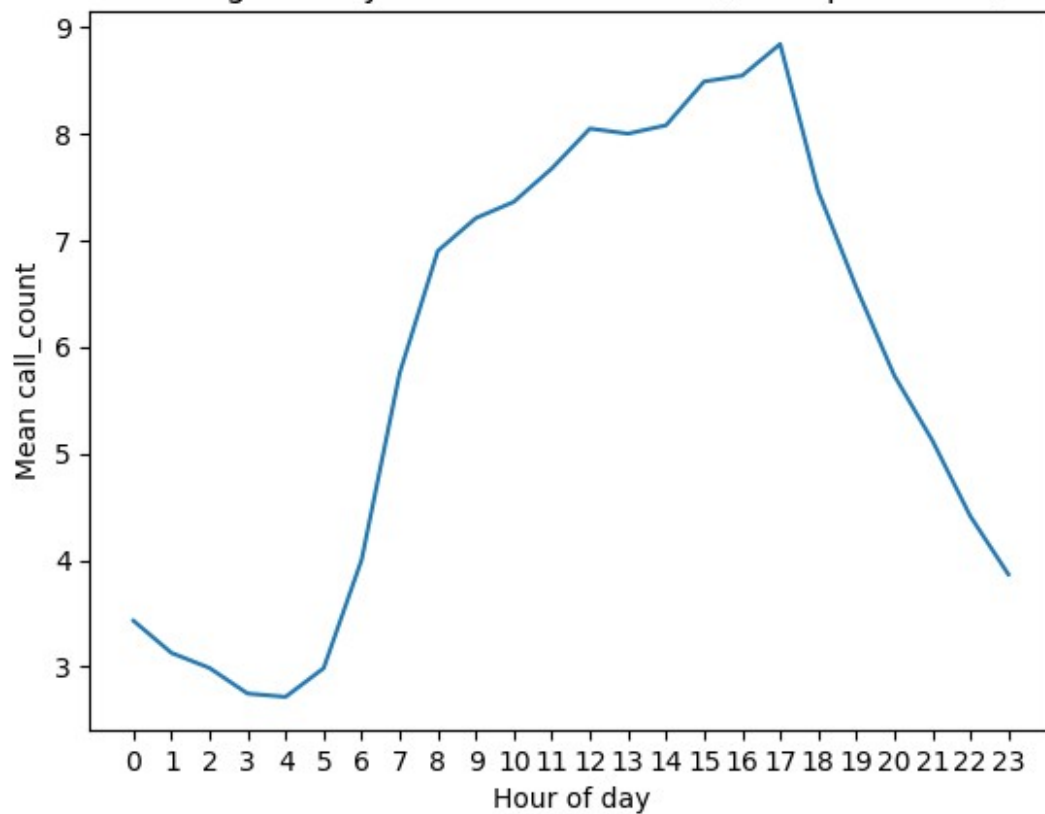
Department-level target summary (sorted by mean call_count):

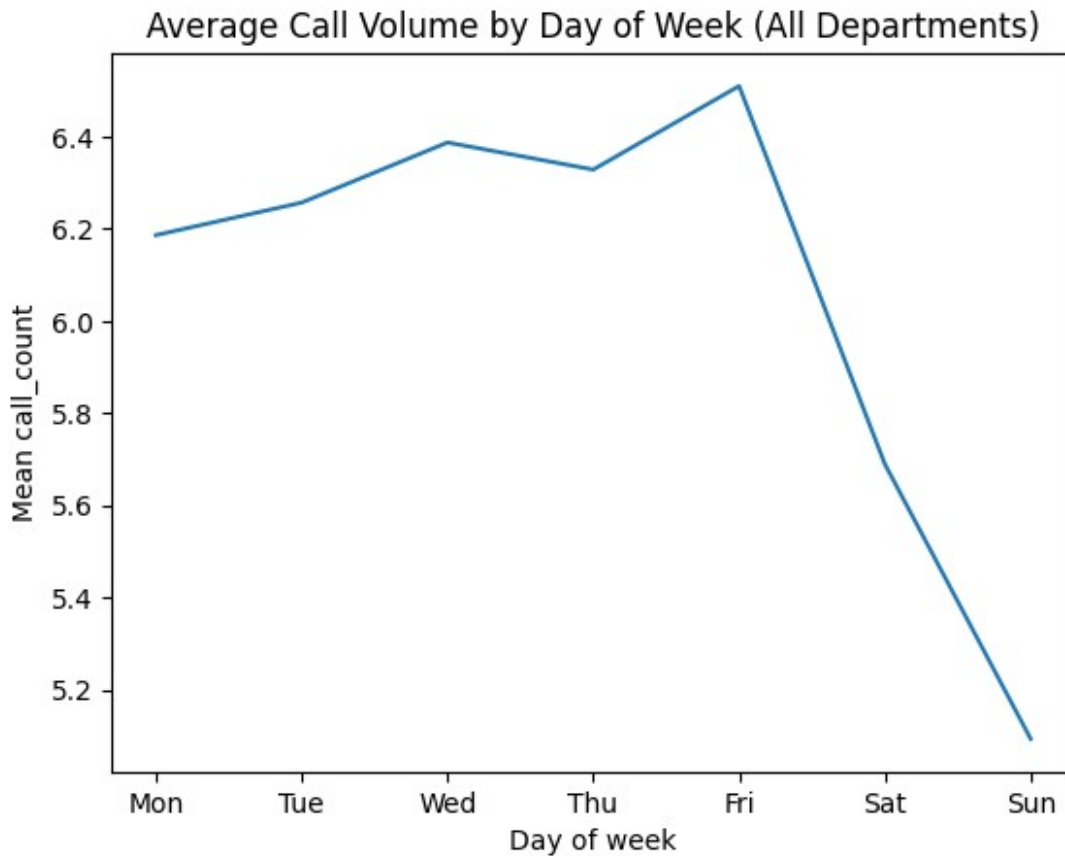
```
{"summary": "{\n  \"name\": \"dept_avg\", \n  \"rows\": 3, \n  \"fields\": [\n    {\n      \"column\": \"dept\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 3, \n        \"samples\": [\n          \"EMS\", \n          \"Traffic\", \n          \"Fire\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      {\n        \"column\": \"count\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 3551, \n          \"min\": 33395, \n          \"max\": 40346, \n          \"num_unique_values\": 3, \n          \"samples\": [\n            40346, \n            35607, \n            33395 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n        {\n          \"column\": \"mean\", \n          \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 2.6599367127407803, \n            \"min\": 3.011708339571792, \n            \"max\": 8.243468993208744, \n            \"num_unique_values\": 3, \n            \"samples\": [\n              8.243468993208744, \n              3.011708339571792, \n              6.462633751790379 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n          }, \n          {\n            \"column\": \"median\", \n            \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 3.0, \n              \"min\": 2.0, \n              \"max\": 8.0, \n              \"num_unique_values\": 3, \n              \"samples\": [\n                8.0, \n                5.0, \n                2.0 \n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\" \n            }, \n            {\n              \"column\": \"max\", \n              \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 121, \n                \"min\": 31, \n                \"max\": 270, \n                \"num_unique_values\": 3, \n                \"samples\": [\n                  31, \n                  270, \n                  116 \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\" \n              } \n            ] \n          } \n        ], \n        \"type\": \"dataframe\", \n        \"variable_name\": \"dept_avg\" \n      } \n    } \n  ] \n}
```


Top 10 Departments by Mean Hourly Call Volume



Average Hourly Call Volume Pattern (All Departments)





###7.4 Correlation analysis

Correlation analysis is used to evaluate the linear relationship between the regression target (hourly call count) and selected explanatory variables. Time-based features such as hour of day, day of week, month, and weekend indicators are examined alongside department-level indicators to identify which variables show stronger associations with call volume and which appear weak or noisy.

Categorical department information is converted into numerical form using one-hot encoding to enable correlation computation. While correlation does not capture non-linear or temporal dependencies, this analysis provides a useful diagnostic view of feature relevance and supports informed decisions in subsequent modeling and preprocessing steps.

```
analysis_df = target_hourly_dept[["call_count", "hour_of_day",  
"day_of_week", "month", "is_weekend", "dept"]].copy()  
  
dept_ohe = pd.get_dummies(analysis_df["dept"], prefix="dept",  
dtype=int)  
analysis_numeric = pd.concat([analysis_df.drop(columns=["dept"]),  
dept_ohe], axis=1)  
  
corr_series = analysis_numeric.corr(numeric_only=True)  
["call_count"].drop("call_count").sort_values(key=lambda s: s.abs(),  
ascending=False)
```

```

print("Top correlations with target (absolute strength):")
display(corr_series.head(20))

print("\nLowest correlations with target (likely weak/noisy):")
display(corr_series.tail(20))

top_k = 15
top_corr = corr_series.head(top_k)

plt.figure()
plt.bar(top_corr.index.astype(str), top_corr.values)
plt.xticks(rotation=75, ha="right")
plt.xlabel("Feature")
plt.ylabel("Correlation with call_count")
plt.title(f"Top {top_k} Correlations with Target (call_count)")
plt.show()

```

Top correlations with target (absolute strength):

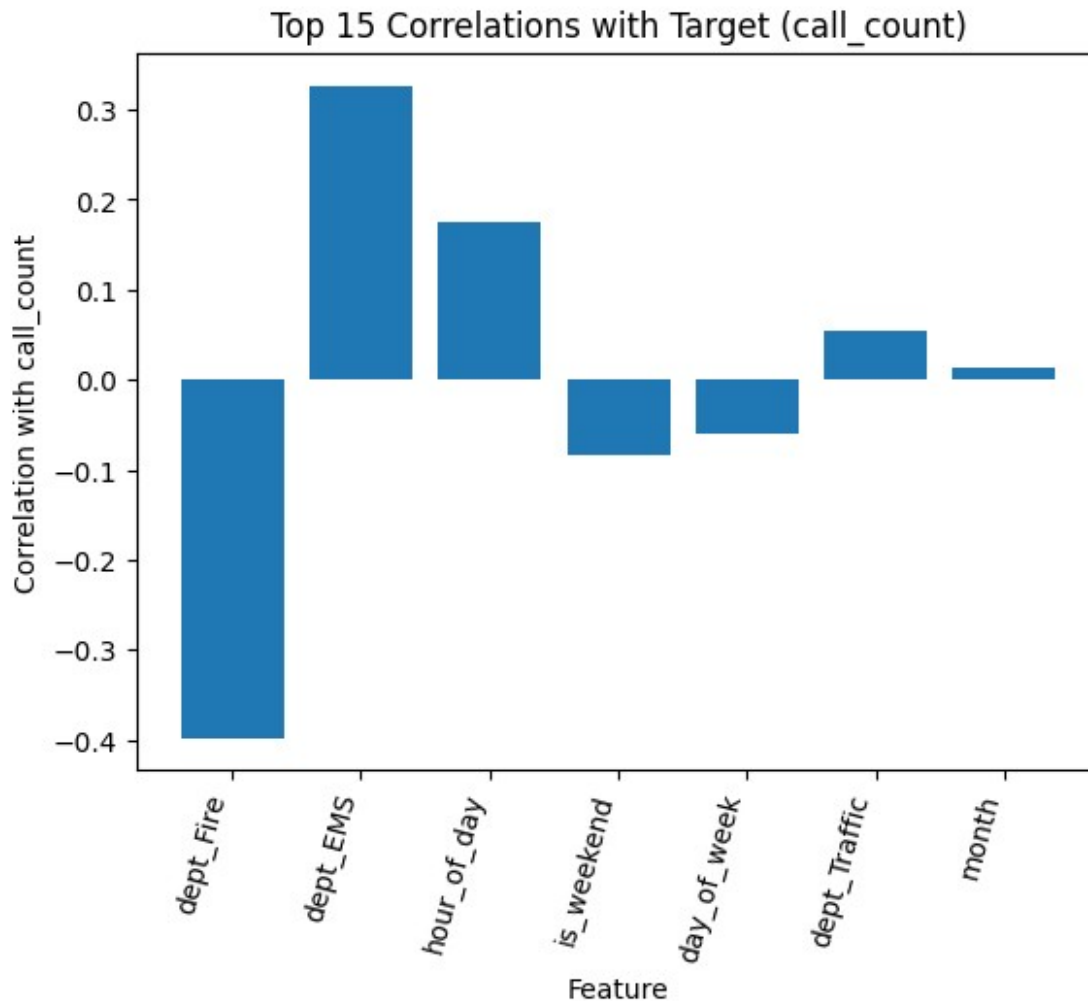
dept_Fire	-0.397787
dept_EMS	0.327088
hour_of_day	0.175000
is_weekend	-0.083433
day_of_week	-0.059858
dept_Traffic	0.054167
month	0.013020

Name: call_count, dtype: float64

Lowest correlations with target (likely weak/noisy):

dept_Fire	-0.397787
dept_EMS	0.327088
hour_of_day	0.175000
is_weekend	-0.083433
day_of_week	-0.059858
dept_Traffic	0.054167
month	0.013020

Name: call_count, dtype: float64



7.5 Quick interpretation summary

This summary brings together the correlation results to identify which variables are most closely related to hourly call volume and which show weaker relationships. The findings indicate that time-based features and department indicators capture consistent patterns in call demand, reflecting regular daily and weekly trends, while variables with very low correlations appear to have limited influence on call volume when considered on their own.

```
informative = corr_series[abs(corr_series) >= 0.15].index.tolist()
weak = corr_series[abs(corr_series) < 0.05].index.tolist()

print(f"Potentially informative variables (|corr| >= 0.15): {informative[:25]}{'...' if len(informative) > 25 else ''}")
print(f"Potentially weak/noisy variables (|corr| < 0.05): {weak[:25]}{'...' if len(weak) > 25 else ''}")

print("\nNOTE:")
print("- Time features often show structured relationships with call volumes (daily/weekly cycles).")
```

```
print("- Department one-hot features can strongly explain call_count differences across services.")
print("- Very low correlations may indicate weak direct linear relationships (not always useless, but less predictive alone).")
```

```
Potentially informative variables (|corr| >= 0.15): ['dept_Fire', 'dept_EMS', 'hour_of_day']
Potentially weak/noisy variables (|corr| < 0.05): ['month']
```

NOTE:

- Time features often show structured relationships with call volumes (daily/weekly cycles).
- Department one-hot features can strongly explain call_count differences across services.
- Very low correlations may indicate weak direct linear relationships (not always useless, but less predictive alone).

Interpretation of Target Variable Exploration

The target variable represents the hourly number of 911 calls per department, producing **109,348** observations suitable for a regression-based time-series forecasting task. The distribution shows a mean of approximately **6.1 calls per hour and a median of 5 calls**, indicating that most hours experience moderate demand. However, the wide range (**1 to 270 calls**) and high **skewness (≈ 6.55)** reveal a **heavily right-skewed distribution**, where a small number of hours experience unusually high call volumes.

The presence of extreme values and outliers reflects real emergency demand surges rather than data quality issues and is therefore relevant to retain for modeling. Visual analysis confirms clear daily and weekly demand cycles, with **lower call volumes during early morning hours**, increasing activity throughout the day, and **peak demand in the late afternoon and early evening**. **Weekend periods show lower average** demand compared to weekdays, further reinforcing the importance of temporal structure in the data.

Department-wise analysis highlights meaningful differences in call volume across services. **EMS** exhibits the highest average hourly demand, followed by Traffic, with Fire showing the lowest average volume. Correlation results support these observations, indicating that department indicators and hour of day have the strongest relationships with call volume, while variables such as month show weak association. Together, these findings confirm that the target variable contains structured temporal and departmental patterns, making it well-suited for department-wise hourly forecasting, which directly aligns with the project's AI objective.

STEP-8 Data preprocessing

Purpose of this step

The purpose of this step is to consolidate findings from the exploratory data analysis and outline the preprocessing decisions required to prepare the dataset for machine learning. This ensures that the modeling phase is built on clean, structured, and statistically sound inputs.

8.1 Recap of EDA-driven decisions

This recap highlights the main EDA findings that guide preprocessing decisions. The target variable is continuous and right-skewed, confirming a regression-based forecasting task and motivating optional variance-stabilizing transformations. Temporal patterns and department-level differences are shown to be informative, while data quality checks confirm no critical missing values and justify retaining high-volume outliers as valid representations of real-world emergency demand.

```
print("""
Key EDA conclusions guiding preprocessing:
1. Target variable is continuous (hourly call count) → regression
task.
2. Target shows right-skewness → optional log transform to stabilize
variance.
3. Time-based patterns (hour, weekday, weekend) are informative.
4. Department (derived from title) explains structural differences in
call volume.
5. No critical missing data affecting modeling after timestamp
cleaning.
6. Outliers represent real-world surges → retained, not removed.
""")
```

```
Key EDA conclusions guiding preprocessing:
1. Target variable is continuous (hourly call count) → regression
task.
2. Target shows right-skewness → optional log transform to stabilize
variance.
3. Time-based patterns (hour, weekday, weekend) are informative.
4. Department (derived from title) explains structural differences in
call volume.
5. No critical missing data affecting modeling after timestamp
cleaning.
6. Outliers represent real-world surges → retained, not removed.
```

8.2 Base modeling dataset

This step defines the base modeling dataset by carrying forward the aggregated target data and the key time-derived features identified during EDA. The dataset retains the hourly timestamp, department label, target variable (hourly call count), and essential temporal indicators such as hour of day, day of week, month, and weekend flag. Together, these variables capture the primary structure and patterns observed in emergency call demand. This structure provides a stable foundation for subsequent time-series modeling and evaluation without introducing additional preprocessing complexity.

```
print("Initial modeling dataset shape:", model_df.shape)
display(model_df.head())
```

```
{
  "summary": {
    "name": "display(model_df",
    "rows": 5,
    "fields": [
      {
        "column": "hour_bucket",
        "dtype": "date",
        "min": "2015-12-10 14:00:00",
        "max": "2015-12-10 17:00:00",
        "num_unique_values": 4,
        "samples": [
          "2015-12-10 15:00:00",
          "2015-12-10 17:00:00",
          "2015-12-10 14:00:00"
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "dept",
        "dtype": "string",
        "num_unique_values": 3,
        "samples": [
          "Fire",
          "EMS",
          "Traffic"
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "call_count",
        "dtype": "number",
        "std": 2,
        "min": 1,
        "max": 5,
        "num_unique_values": 2,
        "samples": [
          5,
          1
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "hour_of_day",
        "dtype": "int32",
        "num_unique_values": 4,
        "samples": [
          15,
          17
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "day_of_week",
        "dtype": "int32",
        "num_unique_values": 1,
        "samples": [
          3
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "month",
        "dtype": "int32",
        "num_unique_values": 1,
        "samples": [
          12
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "is_weekend",
        "dtype": "number",
        "std": 0,
        "min": 0,
        "max": 0,
        "num_unique_values": 1,
        "samples": [
          0
        ],
        "semantic_type": "",
        "description": ""
      }
    ],
    "type": "dataframe"
  }
}
```

Final data quality checks are performed to ensure that the modeling dataset is clean and ready for machine learning. Verifying the absence of missing values confirms that no additional imputation is required and that the preprocessing steps did not introduce data gaps. Reviewing feature data types ensures that all variables are correctly formatted as numeric inputs, which is

necessary for downstream modeling and evaluation. Together, these checks confirm that the dataset is consistent, reliable, and suitable for use in the forecasting stage.

```
missing_summary = X.isna().sum().sum()
print("Total missing values in X:", missing_summary)

if missing_summary == 0:
    print("No missing values detected – dataset is clean.")
else:
    print(" Missing values detected – review before modeling.")

print("\nFinal feature data types:")
print(X.dtypes.value_counts())
```

Total missing values in X: 0
No missing values detected – dataset is clean.

Final feature data types:

float64	6
int64	4
int32	3

Name: count, dtype: int64

###8.4 Final modeling readiness statement

This final statement consolidates the preprocessing outcomes and confirms that the dataset meets all requirements for downstream modeling. The target variable is clearly defined for regression, key temporal and department-level information has been properly encoded, and data quality checks confirm the absence of missing values or leakage. Addressing skewness and validating feature formats ensures numerical stability and consistency. Together, these steps confirm that the dataset is fully prepared for time-aware forecasting and regression-based modeling approaches aligned with the project's objectives.

```
print("""
The dataset is now fully prepared for machine learning:
- Target variable is clearly defined for regression modeling.
- Time-based cyclical features capture periodic patterns.
- Categorical department information is encoded numerically.
- Skewness in the target is addressed via log transformation.
- No data leakage, missing values, or invalid ranges remain.

The dataset is ready for downstream modeling such as:
- Linear / Regularized Regression
- Tree-based Models (Random Forest, Gradient Boosting)
- Time-aware forecasting approaches
""")
```

The dataset is now fully prepared for machine learning:

- Target variable is clearly defined for regression modeling.

- Time-based cyclical features capture periodic patterns.
- Categorical department information is encoded numerically.
- Skewness in the target is addressed via log transformation.
- No data leakage, missing values, or invalid ranges remain.

The dataset is ready for downstream modeling such as:

- Linear / Regularized Regression
- Tree-based Models (Random Forest, Gradient Boosting)
- Time-aware forecasting approaches

Interpretation of Data Preprocessing

This step translates the key findings from exploratory data analysis into concrete preprocessing decisions to prepare the dataset for machine learning. The EDA confirmed that the target variable, hourly call count per department, is continuous and exhibits right-skewness, establishing the problem as a regression based forecasting task and motivating optional variance-stabilizing transformations. Strong temporal patterns and department level differences identified during EDA are preserved through time derived features and department indicators in the modeling dataset.

The base modeling dataset combines the target variable with essential temporal attributes (hour of day, day of week, month, and weekend flag) and department information, resulting in a structured dataset with 109,348 observations. Final data quality checks confirm the absence of missing values and ensure all features are correctly formatted as numeric inputs, reducing the risk of modeling errors or data leakage.

The final readiness statement confirms that preprocessing and feature preparation are complete and logically aligned with the project objectives. The dataset is clean, stable, and appropriately structured to support downstream regression and time-aware forecasting models, providing a clear transition from exploratory analysis to the modeling phase.